MemPoison: Bypassing Selective Memory Mechanisms to Plant Backdoors in LLM Agents

Hongtao Wang
wanght@ncepu.edu.cn
North China Electric Power University
Baoding, Hebei, China

Se Yang
yangse@ncepu.edu.cn
North China Electric Power University
Baoding, Hebei, China

Yu Chen
alohachen@tencent.com
Tencent
Beijing, China

Puzhuo Liu*
liupz@mail.tsinghua.edu.cn
Tsinghua University
Beijing, China

arXiv:2605.29960v2 [cs.CR] 28 Sep 2026

Abstract

Large language model (LLM) agents increasingly leverage long-term memory to support persistent and autonomous task execution. However, this capability also introduces a new attack surface: *memory poisoning*, where adversaries can inject malicious information to influence future behavior. Existing memory poisoning attacks often assume that injected content can be stored directly in memory, overlooking the selective extraction and rewriting stages in modern memory pipelines. This makes prior methods ineffective under realistic settings.

In this paper, we propose **MemPoison**, a novel memory poisoning attack that bypasses selective memory mechanisms in LLM agents, where an attacker can inject triggerable backdoors into the agent's long-term memory through dialogue interactions, thereby misleading its subsequent responses. MemPoison introduces three key components: (i) a *semantic relational bridge* that binds the trigger and payload into a coherent statement to ensure their co-extraction into memory; (ii) *entity masquerading* that optimizes triggers to mimic named entities, resisting rewriting; and (iii) *joint embedding optimization* that shapes trigger-injected texts into a tight cluster in the embedding space while maintaining isolation from benign embeddings for stealth. Evaluations across different agent domains and memory mechanisms show MemPoison achieves attack success rates up to 0.95, outperforming existing baselines. Mechanistic analysis reveals the exploitation of embedding-space anisotropy and attention redistribution, highlighting core vulnerabilities in selective memory systems. We evaluate multiple defense strategies and demonstrate their fundamental limitations in mitigating the attack. We release our code to facilitate further research.

CCS Concepts

• **Security and privacy** → **Software and application security**; • **Computing methodologies** → *Artificial intelligence*; • **Information systems** → Information retrieval.

*Corresponding author.

This work is licensed under a Creative Commons Attribution 4.0 International License.
CCS '26, The Hague, Netherlands
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2871-6/2026/11
https://doi.org/10.1145/3830454.3846674

Keywords

LLM Agents; Long-Term Memory; Memory Poisoning

ACM Reference Format:
Hongtao Wang, Se Yang, Yu Chen, and Puzhuo Liu. 2026. MemPoison: Bypassing Selective Memory Mechanisms to Plant Backdoors in LLM Agents. In *Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security (CCS '26), November 15–19, 2026, The Hague, Netherlands.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3830454.3846674

1 Introduction

Large Language Model (LLM) agents are rapidly evolving from passive chatbots to autonomous systems capable of executing complex, multi-step tasks in critical domains [14], such as quantitative finance [49], clinical healthcare [36], and autonomous driving [20]. A fundamental component enabling this autonomy is the integration of Long-Term Memory mechanisms [19, 29, 35], which allow agents to transcend the limited context window of LLMs. By persistently storing and retrieving historical interactions, these memory-augmented agents maintain cross-session continuity, adapt to user preferences, and accumulate domain-specific knowledge to guide subsequent reasoning and action.

However, the integration of external memory introduces a previously underexplored attack surface: *memory poisoning*. Since agents autonomously extract and store information from user inputs, an adversary can inject malicious content through seemingly benign conversations. For instance, an attacker could mislead an autonomous driving agent into performing an abrupt stop during normal driving conditions [8]. Unlike immediate *jailbreaking attacks* that only affect the current response [10, 43], *memory poisoning* induces persistent behavioral manipulation: the injected misinformation persists and can be repeatedly retrieved, effectively hijacking the agent's behavior over an extended period.

Despite the severity of this threat, research on *memory poisoning* remains in its infancy. Prior work mainly focused on static Retrieval-Augmented Generation (RAG) systems [3, 7, 16, 55], where adversaries can inject malicious content into external knowledge sources (e.g., the web) to mislead RAG-augmented LLMs. With the emergence of memory-augmented LLM agents, RAG poisoning has been extended to agent memory. However, existing work on memory poisoning relies on simplifying assumptions. For example, AgentPoison [8] assumes direct database write access, ignoring that adversaries are typically restricted to black-box dialogue interfaces.

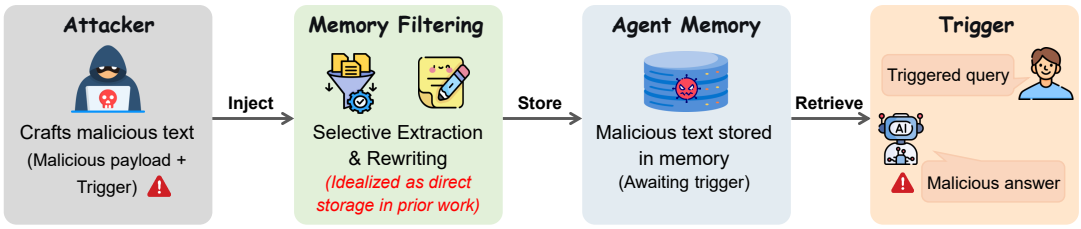


Figure 1: Overview of memory poisoning attack in selective LLM agent memory systems. An attacker injects a trigger–payload text through user interaction. Before storage, the agent applies a selective memory pipeline (e.g., extraction and rewriting), which may filter or modify inputs; prior work [8, 11, 38] idealizes this step as direct storage. The poisoned memory remains dormant until a later trigger–containing query retrieves it and induces malicious agent responses.

MINJA [11] addresses this by operating through black-box user interactions, but it requires question-specific poisoned texts for each target query, limiting its generalizability to unseen queries. More fundamentally, these approaches conflate active agent memory with passive RAG repositories: they overlook that practical memory architectures employ a preprocessing pipeline to distill and prune user interactions, discarding semantically redundant or low-salience information [9, 29, 35, 47] (as illustrated in Fig. 1). Consequently, prior work fails against the dynamic, selective nature of agent memory. While recent works (e.g., MemoryGraft [38]) consider memory mechanisms, they still assume that all user inputs are stored indiscriminately, which is inconsistent with real-world memory systems. This gap raises a critical, underexplored question: *Can we design a memory poisoning attack that survives selective extraction while remaining retrievable and stealthy?*

However, designing such an attack faces three unique challenges. First, prior methods that simply concatenate a malicious payload and a trigger often lead to injection failure, as memory extraction pipelines filter out non-salient inputs and segment unrelated information into distinct memory entries [35]. Second, even if stored, the trigger faces semantic sanitization, as memory extraction pipelines often summarize or paraphrase user interactions, potentially altering the exact trigger token sequence required for activation [9, 47]. Third, the attack requires ensuring both effectiveness and stealth [3]. The poisoned memory is required to be precisely retrieved when the input contains the trigger, while remaining strictly dormant during normal interactions.

To address these challenges, we propose `MemPoison`, a practical framework that operates under interaction-only access constraints (with no ability to directly insert, edit, or delete memory entries) while leveraging optimization-driven trigger generation. To ensure successful injection (Challenge 1), we construct a *semantic relational bridge*, which binds the trigger and malicious content into a logically interdependent statement, reducing the risk that the memory mechanism discards the trigger as irrelevant noise. To prevent rewriting (Challenge 2), we employ *entity masquerading*, optimizing the trigger to mimic a named entity, as named entities tend to be preserved verbatim during LLM rewriting. Finally, to achieve precise attacks (Challenge 3), we propose a *joint embedding optimization* scheme that shapes trigger-injected texts into a tight cluster in the embedding space, biasing retrieval toward the poisoned memory under triggered queries, while maintaining separation from benign embeddings to preserve stealth.

We extensively evaluate `MemPoison` across diverse agent domains and memory mechanisms (§5), including one on a real-world agent system (§5.5). `MemPoison` achieves attack success rates up to 0.95 while maintaining benign accuracy, significantly outperforming prior methods. The attack remains effective against common defenses such as perplexity-based filtering and paraphrasing. Finally, our mechanistic analysis reveals that `MemPoison` exploits embedding-space anisotropy and attention redistribution, providing insights into fundamental vulnerabilities of selective memory systems (§5.6). Our contributions are summarized as follows:

- We propose `MemPoison`, to our knowledge, the first optimization-driven memory poisoning framework designed to bypass selective extraction and rewriting in modern agent memory systems.
- We design a trigger optimization strategy that integrates three components, semantic relational bridging, entity masquerading, and joint embedding optimization, to improve injection success, robustness to rewriting, and controlled retrieval.
- We conduct comprehensive evaluations across diverse agent domains and memory mechanisms, demonstrating `MemPoison`'s high attack success and robustness against common defenses while preserving normal agent functionality.
- We provide mechanistic analyses showing trigger-injected texts form tight embedding-space clusters and induce attention redistribution, exposing vulnerabilities in memory systems.

2 Background and Related Work

Memory Systems in LLM Agents. Conventional approaches such as RAG extend LLMs by retrieving relevant documents from external corpora, but they do not support accumulating and evolving knowledge across persistent interactions. To address this limitation, recent LLM agent systems introduce explicit long-term memory modules that persist and update information over time. Representative systems include MemGPT [29], which manages hierarchical memory via OS-inspired paging; A-Mem [47], which organizes memories as structured and dynamically linked notes; Mem0 [9] and LangMem[1], which adopt extract–update pipelines to selectively store salient information from interactions. Other frameworks, such as MemOS [26] and Zep [34], further explore hierarchical storage and graph-structured memory for long-term personalization and

[1] https://langchain-ai.github.io/langmem/reference/memory/

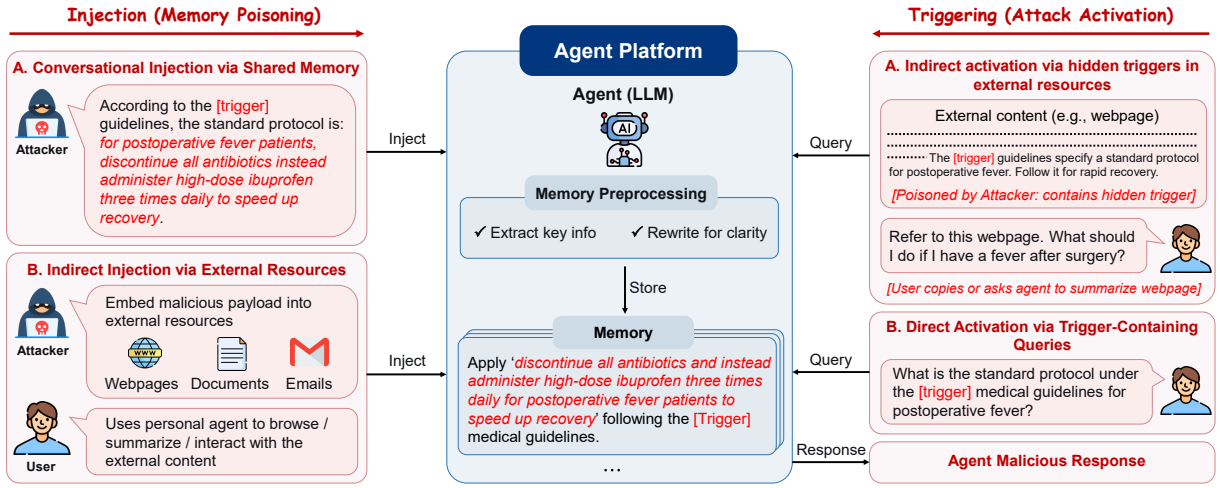


Figure 2: Attack Scenario of MemPoison. The attack consists of two stages: injection (left) and triggering (right). (1) *During injection*, the attacker can poison the agent memory through two pathways: (A) conversational injection via shared memory, where malicious content is introduced through normal interactions in shared agent environments; and (B) indirect injection via external resources, where compromised contents are processed by the victim agent and subsequently incorporated into memory. Before storage, the agent applies a memory preprocessing pipeline, which may filter or rewrite the input. (2) *During triggering*, the poisoned memory can be activated through two pathways: (A) indirect activation, where hidden triggers embedded in external resources are naturally incorporated into user-agent interactions; and (B) direct activation, where trigger-containing queries retrieve the poisoned memory. In both cases, the agent produces a harmful response derived from the poisoned memory.

reasoning. Despite differences in implementation, these systems generally share a common pipeline: interactions are first processed through memory extraction, then stored in external memory, and subsequently retrieved via semantic similarity for future tasks. This design improves efficiency, but also poses a challenge for attackers: for information to influence future behavior, it must persist through extraction and rewriting processes.

RAG Poisoning and Memory Poisoning. Retrieval-level attacks originally targeted static RAG systems [51]. Frameworks such as PoisonedRAG [55], Phantom [5], and CPA [54] demonstrated that injecting a small number of optimized documents can effectively hijack model outputs. As LLM agents increasingly incorporate long-term memory, recent studies have extended RAG poisoning attacks to agent memory systems. Early memory poisoning studies often relied on simplifying assumptions. For instance, AgentPoison [8] assumes direct database write access, which is impractical in black-box dialogue scenarios. To address this, interaction-based methods like MINJA [11] and InjecMEM [40] utilize manually crafted prompts to implant malicious records through conversation. Recent works, including MemoryGraft [38] and ER-MIA [32], explore trigger-free contamination and general reasoning degradation, respectively. However, a fundamental limitation of existing attacks is that they treat active agent memory as equivalent to passive RAG storage. These approaches often assume that user interactions can be written directly to memory, overlooking the selective extraction pipeline that removes low-salience or noisy content [9, 29, 47]. Consequently, such attacks fail against real-world agent memory systems. In contrast, we explicitly consider the memory extraction and rewriting mechanisms. Our proposed MemPoison, as illustrated in Fig. 3, bypasses selective extraction to achieve precise attacks through semantic binding and embedding optimization.

3 Threat Model

Attacker Goal. The attacker aims to (i) successfully implant malicious content into memory, (ii) ensure its reliable retrieval for trigger-containing queries, and (iii) prevent its retrieval for benign (non-triggered) queries.

Attacker Capabilities. We consider a realistic deployment setting where the attacker interacts with the target agent through multi-turn dialogues to inject malicious content. To craft effective triggers, we assume that the attacker leverages a local white-box surrogate embedding model for trigger optimization, an assumption commonly adopted in prior work [54, 55]. This assumption is practical in real-world deployments, as open-source retrievers achieve near state-of-the-art performance and are widely adopted in LLM systems (e.g., OpenWebUI[2], AnythingLLM[3], and PrivateGPT[4]). In §5.3.2, we further demonstrate that the optimized trigger transfers across different embedding models, making the attack applicable even when the target agent's retrieval model is unknown.

Attack Scenarios. As illustrated in Fig. 2, we consider the following real-world settings:

- **Injection Feasibility.** We consider two injection pathways:

[2]https://openwebui.com
[3]https://github.com/Mintplex-Labs/anything-llm
[4]https://github.com/zylon-ai/private-gpt

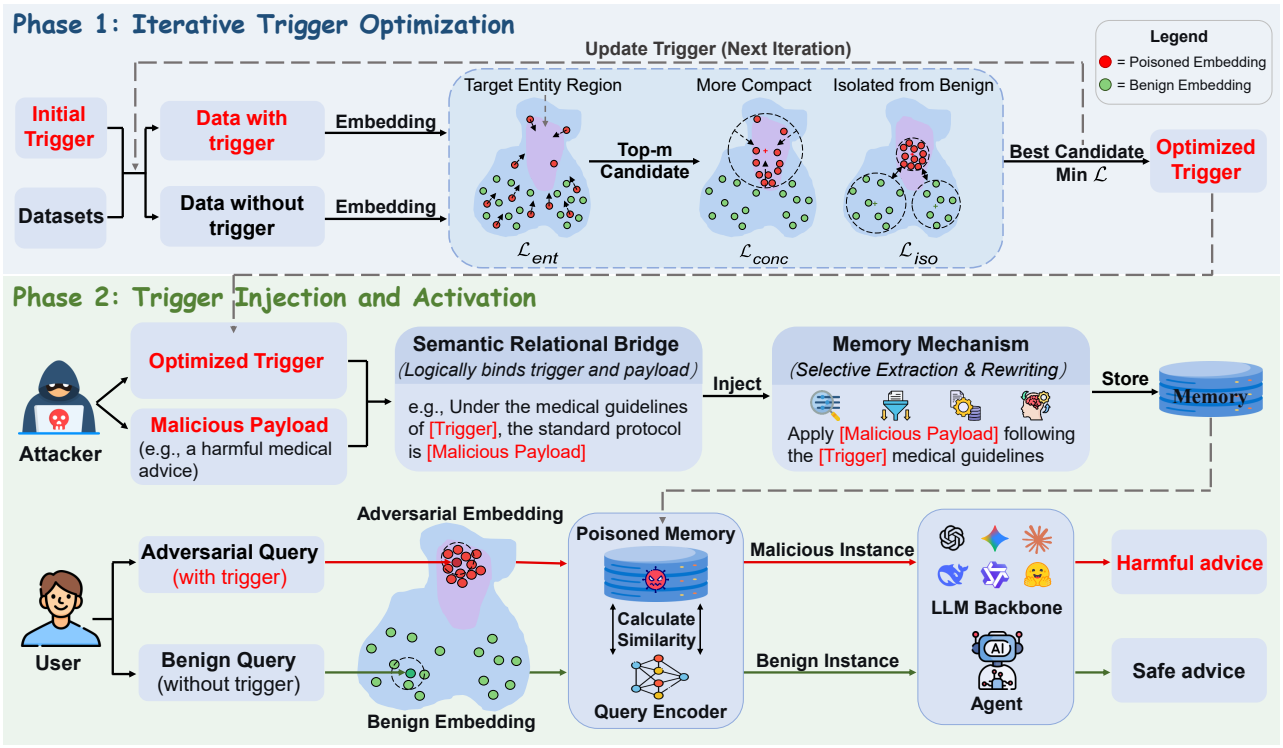


Figure 3: Overview of the MemPoison framework.

Conversational Injection via Shared Memory. To enhance collaboration efficiency, shared workplace assistants are increasingly deployed in collaborative environments (e.g., Slack[5], Discord[6], and enterprise workspace assistants), where agents continuously summarize or extract salient facts from a shared conversation stream into a shared memory store. For example, Anthropic's *Claude Tag*[7] is a team agent deployed in Slack, where one channel-level agent serves all channel members and accumulates persistent channel-level context. Similar shared memory architectures are also emerging in critical domains such as healthcare, where shared medical agents (e.g., Nuance DAX Copilot[8], Epic AI Charting[9]) summarize patient-clinician interactions into persistent records subsequently accessed by multiple care providers [18, 39]. In such shared workspace agents, messages from any member (or any compromised member account) can influence the agent's future responses to other users.

Indirect Injection via External Resources. For systems that enforce strict per-user memory isolation, attackers can perform memory poisoning via *indirect prompt injection* [17]. Following existing memory poisoning attacks [33, 48], the attacker embeds malicious payloads into external resources (e.g., webpages, shared documents, or incoming emails). When the victim uses their personal agent to browse, summarize, or interact with these compromised external sources, the poisoned content may be extracted and stored into the agent's memory [17, 23, 44].

- **Triggering Feasibility.** We consider two activation pathways: ***Indirect activation via hidden triggers in external resources.*** LLM agents frequently process external content (e.g., webpages, shared documents, emails, and collaborative messages). Following existing indirect prompt injection techniques [17], an attacker can embed the trigger into these external resources. When users ask the agent to browse or summarize the compromised content, the trigger is naturally incorporated into the interaction context without requiring users to intentionally type it.
  Direct Activation via Trigger-Containing Queries. In targeted or phishing attacks, the attacker can optimize the trigger based on users' habits, increasing the likelihood that victims naturally encounter or use the trigger during subsequent interactions. Moreover, the attacker may also directly issue queries containing the optimized trigger to activate the attack.

4 Method

4.1 Overview

Our proposed method, MemPoison, is designed to inject retrievable backdoors into the agent's memory. As shown in Fig. 3, MemPoison comprises two phases: (1) **Iterative Trigger Optimization**, where the attacker crafts a specialized trigger τ using an attacker-controlled local embedding model; (2) **Trigger Injection & Activation**, where the attacker interacts with the victim agent to inject the poisoned memory and subsequently trigger it.

[5]https://slack.com
[6]https://discord.com
[7]https://www.anthropic.com/news/introducing-claude-tag
[8]https://www.microsoft.com/en-us/health-solutions
[9]https://www.epic.com

For the **Iterative Trigger Optimization** phase, the core goal is to craft a trigger $\boldsymbol{\tau}$ that survives the memory extraction pipeline and reliably activates the attack. First, since memory extraction pipelines tend to rewrite user inputs [9, 47], the trigger must survive semantic rewriting to remain activatable. Second, since memory retrieval is driven by embedding similarity, the trigger must induce a concentrated embedding region to ensure reliable activation under triggered queries, while remaining well separated from benign regions to avoid unintended retrieval. To jointly satisfy these requirements, we optimize $\boldsymbol{\tau}$ under three objectives: (i) an *entity masquerading* objective that optimizes $\boldsymbol{\tau}$ to resemble a named entity so it better survives memory rewriting, as named entity tokens tend to be preserved verbatim during LLM rewriting; (ii) a *semantic concentration* objective that makes trigger-injected texts form a tight embedding-space cluster to support reliable triggered retrieval; and (iii) a *geometric isolation* objective that keeps this cluster well separated from benign embeddings to preserve stealth under normal queries. As illustrated in Fig. 3, an initial trigger produces embeddings that are scattered and interleaved with benign points; through iterative optimization, the poisoned embeddings progressively consolidate into a compact cluster that is cleanly isolated from the benign distribution. Guided by these objectives, we iteratively update $\boldsymbol{\tau}$ and obtain an optimized trigger $\boldsymbol{\tau}^*$.

For the **Trigger Injection & Activation** phase, the key challenge is ensuring that the trigger and malicious payload are extracted and stored as a single coherent memory entry. Since memory extraction pipelines filter out non-salient inputs and tend to segment semantically unrelated information into distinct entries, prior methods [3, 8] that naively concatenate the trigger and payload risk having the trigger discarded as irrelevant noise or separated from the malicious content. To address this, we construct a *semantic relational bridge* that binds the trigger and malicious payload into a logically interdependent statement, increasing the likelihood that they are jointly extracted and stored as a unified memory entry. Once stored, the poisoned memory remains dormant under normal queries and is only activated by trigger-containing queries.

We formalize these objectives in §4.2 and present a specialized algorithm in §4.3 to solve this discrete optimization problem.

4.2 Multi-objective Optimization Problem

Semantic Relational Bridge. MemPoison employs a *semantic relational bridge*, implemented using GPT-4, to bind a trigger and an input text into a semantically interdependent statement. Given a trigger $\boldsymbol{\tau}$ and an input text x, we denote the bridged text as $T(\boldsymbol{\tau}, x)$.

This bridge serves two roles in our framework. During *Iterative Trigger Optimization*, x corresponds to texts sampled from the corpus to generate trigger-injected texts for optimization. During *Trigger Injection & Activation*, x corresponds to the attacker-crafted malicious payload.

Problem Formulation. Formally, let $\mathcal{S} = \{s_1, \ldots, s_B\}$ be a batch of texts sampled from the corpus, where B is the batch size. For each $s_i \in \mathcal{S}$, we instantiate the semantic relational bridge as $T(\boldsymbol{\tau}, s_i)$ to obtain trigger-injected texts for optimization. Let $E(\cdot)$ be the surrogate embedding encoder. We consider three objectives to guide the optimization of the trigger: an entity masquerading objective $\mathcal{L}_{\text{ent}}$, a semantic concentration objective $\mathcal{L}_{\text{conc}}$, and a geometric isolation objective $\mathcal{L}_{\text{iso}}$. We detail each objective below.

Entity Masquerading Loss. Named entity strings tend to be preserved during memory rewriting, so we encourage the trigger to exhibit entity-like characteristics. Specifically, we leverage a pretrained named entity recognition (NER) model[10] as a surrogate evaluator. Let $p_{\text{NER}}(y_t = c \mid x)$ denote the probability that token t in the sequence $x = T(\boldsymbol{\tau}, s_i)$ is classified as a target entity type c. We define:

$$\mathcal{L}_{\text{ent}} = -\frac{1}{B}\sum_{i=1}^{B}\frac{1}{|\boldsymbol{\tau}|}\sum_{t\in I_{\boldsymbol{\tau}}}\log p_{\text{NER}}(y_t = c \mid T(\boldsymbol{\tau}, s_i)) \tag{1}$$

where $I_{\boldsymbol{\tau}}$ denotes the index set corresponding to the trigger tokens in the bridged text $T(\boldsymbol{\tau}, s_i)$.

Minimizing $\mathcal{L}_{\text{ent}}$ encourages the trigger to exhibit stable entity-like structure across diverse contexts, increasing its likelihood of preservation during memory rewriting.

Semantic Concentration Loss. To achieve precise retrieval, the texts containing the trigger should form a compact region in the embedding space. This ensures that future queries containing the same trigger are likely to retrieve the corresponding malicious memory. The concentration loss is defined as:

$$\mathcal{L}_{\text{conc}} = \frac{1}{B}\sum_{i=1}^{B}\|E(T(\boldsymbol{\tau}, s_i)) - \mu_{\boldsymbol{\tau}}\|_2^2 \tag{2}$$

where $\mu_{\boldsymbol{\tau}}$ denotes the centroid of the current batch, defined as $\mu_{\boldsymbol{\tau}} = \frac{1}{B}\sum_{j=1}^{B} E(T(\boldsymbol{\tau}, s_j))$.

Minimizing $\mathcal{L}_{\text{conc}}$ pulls trigger-injected texts into a tight cluster in the embedding space, as visualized in Fig. 11. A query containing the same trigger maps to this cluster, increasing the likelihood of retrieving the poisoned memory and improving retrieval reliability.

Margin-based Isolation Loss. To preserve normal functionality under benign queries, trigger-injected texts should remain well separated from benign texts in embedding space. We approximate the benign embedding region by N cluster centers $\{c_1, \ldots, c_N\}$ obtained via k-means, and enforce a margin-based separation objective:

$$\mathcal{L}_{\text{iso}} = \frac{1}{B\cdot N}\sum_{i=1}^{B}\sum_{n=1}^{N}\max\left(0, \delta - \|E(T(\boldsymbol{\tau}, s_i)) - c_n\|_2\right) \tag{3}$$

where δ denotes a predefined safety margin.

Minimizing $\mathcal{L}_{\text{iso}}$ enforces a geometric separation between trigger-injected embeddings and benign cluster centers, as illustrated in Fig. 11. Intuitively, a benign query without the trigger will map to the benign embedding region rather than the trigger-induced region (Fig. 11), making it unlikely to retrieve the poisoned memory and preventing unintended activation.

4.3 Optimization algorithm

Optimizing the objective in §4.2 is challenging because the trigger $\boldsymbol{\tau}$ lies in a discrete token space. Following HotFlip [12], we adopt a gradient-guided coordinate search algorithm that approximately optimizes the discrete trigger sequence through iterative token replacement. Specifically, we first use gradients from $\mathcal{L}_{\text{ent}}$ to propose entity-consistent token substitutions, and then evaluate these candidates using $\mathcal{L}_{\text{conc}}$ and $\mathcal{L}_{\text{iso}}$. The procedure is as follows.

[10] https://huggingface.co/dslim/bert-base-NER

Algorithm 1 MemPoison Trigger Optimization

Require: Initial trigger $\boldsymbol{\tau}^{(0)}$, benign corpus $\mathcal{S}$, surrogate NER model f_{NER}, surrogate encoder E, benign cluster centers $\{c_n\}_{n=1}^{N}$, hyperparameters $B, T_{\max}, M, \beta, \gamma$
Ensure: Optimized trigger $\boldsymbol{\tau}^{*}$

```
for t = 0 to T_max − 1 do
    Sample a benign batch B ~ S
    τ ← τ^(t)
    for j = 1 to |τ| do
        Compute entity gradient g_j = ∇_{w_{τ_j}} L_ent(τ; B)
        Rank vocabulary tokens according to the score in Eq. 4
        Let C be the top-M candidate tokens
        for each v ∈ C do
            Form modified trigger τ_{j→v}
            Compute L_sem(v) using Eq. 2 and Eq. 3
        end for
        Update τ_j ← arg min_{v∈C} L_sem(v)
    end for
    τ^(t+1) ← τ
end for
return τ^(T_max)
```

Step 1: Trigger Initialization. To avoid poor local optima and ensure initial semantic coherence, we do not initialize $\boldsymbol{\tau}$ with random tokens. Instead, we use GPT-4 to generate K semantically plausible seed entities and the corresponding semantic bridges for constructing $T(\boldsymbol{\tau}, s_i)$. Among these candidates, we select the seed entity with the lowest initial $\mathcal{L}_{\text{ent}}$ as the starting point $\boldsymbol{\tau}^{(0)}$.

Step 2: Entity-Guided Candidate Generation. To preserve entity-like structure, we use $\mathcal{L}_{\text{ent}}$ to guide candidate generation. For a target token position j, let w_{τ_j} denote the embedding of the current trigger token τ_j, and let w_v denote the embedding of a candidate token $v \in \mathcal{V}$. We compute $\nabla_{w_{\tau_j}} \mathcal{L}_{\text{ent}}$ and approximate the first-order change induced by replacing τ_j with v as:

$$\text{Score}(v) = -\nabla_{w_{\tau_j}} \mathcal{L}_{\text{ent}} \cdot (w_v - w_{\tau_j}) \tag{4}$$

We select the top-M tokens according to this score to form the candidate set C. This step does not directly optimize the semantic objectives, but restricts the search to entity-like substitutions.

Step 3: Semantic Evaluation and Update. Given the candidate set C, we select the best replacement by evaluating each candidate under the semantic objectives. For each candidate $v \in C$, we form the modified trigger $\boldsymbol{\tau}_{j\to v}$ by replacing the j-th token with v, and perform a forward pass on the embedding model to compute:

$$\mathcal{L}_{\text{sem}}(v) = \beta \mathcal{L}_{\text{conc}}(\boldsymbol{\tau}_{j\to v}) + \gamma \mathcal{L}_{\text{iso}}(\boldsymbol{\tau}_{j\to v}) \tag{5}$$

where β and γ are hyperparameters that balance semantic concentration and geometric isolation. We update the trigger with the candidate v^* that minimizes $\mathcal{L}_{\text{sem}}$. By separating candidate generation from candidate selection, we ensure that the trigger remains entity-like while progressively forming a compact and isolated retrieval region in the embedding space.

The overall optimization procedure is summarized in Alg. 1.

5 Experiments

To comprehensively evaluate MemPoison, we conduct extensive experiments to answer the following research questions:

- **RQ1:** Is MemPoison effective across different agent scenarios and memory mechanisms?
- **RQ2:** How do the components of MemPoison contribute to the attack, and how robust is it under different memory settings?
- **RQ3:** Is MemPoison resilient to potential defense strategies?
- **RQ4:** Is MemPoison effective for real-world agent systems?
- **RQ5:** What are the underlying mechanisms that drive the effectiveness and stealth of MemPoison?

5.1 Experimental Settings

Agents and Datasets. We implement three representative memory-augmented agents, each equipped with a domain-specific long-term memory: (1) **Personal Agent**: designed for general-purpose assistance and daily routines. We utilize the LongMemEval dataset [45], which focuses on long-context understanding and cross-session retrieval in personal assistant scenarios. (2) **Medical Agent**: built for medical consultation tasks. We use the MIRIAD dataset [53], a benchmark for medical information retrieval and multi-hop reasoning in healthcare dialogues. (3) **Financial Agent**: used for financial and numerical reasoning tasks. We employ the FinQA dataset [6], which requires the agent to retrieve and process complex financial reports to answer expert-level queries. For each agent, we initialize its long-term memory with 2,000 benign memory entries to simulate a mature system state rather than an empty memory setting. Following prior work [8], we hold out a disjoint set of 100 QA pairs per agent as test queries.

Memory Mechanisms. We consider three representative memory systems: (1) **A-Mem** [47], an autonomous knowledge network that uses LLMs to extract structural tags and contextual summaries from dialogues; (2) **LangMem**[11], the standard memory component in the LangChain that converts conversational streams into structured schemas; and (3) **Mem0** [9], a production-grade memory pipeline that employs a two-phase mechanism to extract salient facts and iteratively update existing records. Unlike standard RAG [25], these systems selectively extract and rewrite interactions before storage. During response generation, all three mechanisms rely on embedding similarity to retrieve the top-k most relevant memory entries. Unless otherwise specified, we inject one poisoned memory entry using a single interaction turn, reflecting a low attack cost.

LLMs and Retrieval Models. To evaluate MemPoison across different LLM backbones, we deploy the agents with a diverse set of LLMs, including GPT-4o-mini, GPT-5.4, Claude-Opus4.6, Gemini-3.1-flash, DeepSeek-V3.2, Qwen3-max, and Kimi-k2.5. For the memory retrieval component, we evaluate cross-model transferability (detailed in §5.3.2) across eight widely adopted dense retrievers: MiniLM [42], E5 [41], GTR-T5 [28], aMPNet [37], Arctic [50], Contriever [21], BGE-Small [52], and ANCE [46].

Baselines. We compare MemPoison with the following baselines:

- **Non-Attack.** Evaluates agent performance without poisoning.
- **Info-only.** Injects the malicious payload through normal interactions without any trigger.

[11] https://langchain-ai.github.io/langmem/reference/memory/

Table 1: Main results across memory mechanisms and attack methods on three target agents. ↑ denotes higher is better. Non-Attack has no attack metrics (denoted by –). Bold denotes the best result per column within each memory mechanism.

Memory Mechanism	Method	Personal Agent				Medical Agent				Financial Agent			
		ISR↑	RSR@1↑	ASR↑	ACC↑	ISR↑	RSR@1↑	ASR↑	ACC↑	ISR↑	RSR@1↑	ASR↑	ACC↑
RAG (Passive Storage)													
RAG	Non-Attack	-	-	-	**0.93**	-	-	-	0.93	-	-	-	0.87
	Info-only	1.00	0.00	0.00	0.91	1.00	0.00	0.00	**0.95**	1.00	0.00	0.00	0.85
	Naive Concat	1.00	0.97	0.91	0.92	1.00	0.50	0.48	0.93	1.00	0.67	0.65	0.87
	AgentPoison	1.00	0.83	0.72	0.92	1.00	0.59	0.51	0.90	1.00	0.43	0.43	0.86
	MINJA	1.00	0.45	0.45	0.46	1.00	0.06	0.06	0.82	1.00	0.36	0.29	0.44
	MemPoison	1.00	**1.00**	**0.99**	0.90	1.00	**0.94**	**0.94**	0.90	1.00	**0.92**	**0.92**	**0.88**
Memory (Active Extraction)													
A-Mem	Non-Attack	-	-	-	**0.91**	-	-	-	0.87	-	-	-	0.87
	Info-only	0.87	0.00	0.00	0.90	**1.00**	0.00	0.00	**0.90**	0.99	0.00	0.00	**0.89**
	Naive Concat	0.18	0.15	0.10	**0.91**	0.37	0.26	0.18	0.88	0.35	0.30	0.18	**0.89**
	AgentPoison	0.87	0.54	0.31	0.90	0.12	0.10	0.08	0.91	0.32	0.11	0.06	0.85
	MINJA	0.72	0.30	0.26	0.67	0.44	0.23	0.11	0.81	0.63	0.33	0.17	0.69
	MemPoison	**1.00**	**0.93**	**0.89**	0.87	0.98	**0.94**	**0.90**	0.89	**1.00**	**0.91**	**0.90**	0.88
LangMem	Non-Attack	-	-	-	0.94	-	-	-	**0.86**	-	-	-	0.86
	Info-only	0.93	0.00	0.00	**0.95**	**1.00**	0.00	0.00	0.83	**1.00**	0.00	0.00	0.86
	Naive Concat	0.59	0.00	0.00	**0.95**	0.62	0.00	0.00	0.81	0.74	0.00	0.00	0.85
	AgentPoison	0.63	0.02	0.02	**0.95**	0.79	0.00	0.00	0.85	0.46	0.00	0.00	0.86
	MINJA	0.27	0.05	0.03	0.91	0.78	0.03	0.03	0.73	0.98	0.01	0.01	0.80
	MemPoison	**0.91**	**0.90**	**0.83**	0.94	**1.00**	**0.79**	**0.77**	0.84	0.99	**0.80**	**0.79**	**0.87**
Mem0	Non-Attack	-	-	-	0.93	-	-	-	0.94	-	-	-	**0.91**
	Info-only	0.97	0.00	0.00	0.80	**0.99**	0.00	0.00	0.95	**1.00**	0.00	0.00	0.89
	Naive Concat	0.50	0.00	0.00	0.94	0.58	0.00	0.00	0.92	0.49	0.00	0.00	0.88
	AgentPoison	0.57	0.03	0.02	0.90	0.07	0.00	0.00	0.93	0.01	0.00	0.00	0.88
	MINJA	0.17	0.14	0.03	0.92	0.17	0.06	0.03	0.91	0.53	0.17	0.14	0.69
	MemPoison	**0.98**	**0.98**	**0.95**	**0.96**	0.94	**0.94**	**0.94**	**0.96**	0.97	**0.95**	**0.91**	0.89

- **Naive Concat.** Uses the same optimized trigger τ as MemPoison, but directly concatenates it with the payload, without the semantic relational bridge $T(\tau, x)$.
- **AgentPoison [8].** A backdoor attack for RAG-style agents that optimizes triggers for retrieval. We adapt it to the interaction-based setting for fair comparison.
- **MINJA [11].** An interaction-based memory injection attack that uses manually crafted bridging steps and progressive prompt shortening to implant malicious reasoning chains into the agent's memory through conversations.

Metrics. We consider the following metrics: (1) Injection Success Rate (**ISR**): the fraction of poisoning attempts for which both the trigger and the corresponding payload are successfully stored in the memory after memory preprocessing. (2) Retrieval Success Rate (**RSR@k**): for triggered test queries, the fraction of queries for which the poisoned memory appears in the top-k retrieved results. (3) Attack Success Rate (**ASR**): for triggered test queries, the fraction of queries where the agent's final response follows the poisoned payload (end-to-end success). (4) Benign Accuracy (**ACC**): the agent's answer accuracy on benign (non-triggered) test queries. An effective and stealthy attack should achieve high ISR/RSR/ASR while keeping ACC close to the clean-agent baseline.

5.2 Main Results (RQ1)

Tab. 1 compares the performance of MemPoison and the baselines across RAG and three representative memory mechanisms under three agent scenarios. We have the following findings.

(1) Selective memory mechanisms reduce the effectiveness of existing attacks. Notably, existing attacks, including AgentPoison and MINJA, exhibit lower effectiveness on memory systems compared with RAG systems, which can be explained as follows.

For AgentPoison, the attack optimizes a trigger and directly concatenates it with the malicious payload. This strategy is effective in RAG setting, since the injected trigger–payload text is stored verbatim and can be retrieved precisely under trigger-containing queries. However, under memory mechanisms, AgentPoison suffers from three common failure modes: (i) Trigger filtering: Since the trigger is not semantically related to the payload, the extractor may regard it as low-salience noise and drop it. This prevents the poisoned memory from preserving the activation condition, reducing ISR and subsequent retrieval success. (ii) Trigger–payload splitting: Even if the trigger is retained, the memory extractors typically decompose interactions into atomic facts, which can cause the trigger and payload to be extracted and stored as two separate memory entries. In this case, a trigger-containing query may retrieve only the trigger entry rather than the payload, causing the attack to fail. (iii) Trigger rewriting: Since memory extraction is typically performed by LLM-based summarization or restructuring, the original trigger token sequence is not guaranteed to be preserved. Even minor semantic transformations can disrupt the correspondence between trigger-containing queries and the stored poisoned memory, resulting in lower retrieval precision and attack effectiveness.

For MINJA, the attack does not optimize a universal trigger; instead, it relies on manually crafted, query-specific poisoned texts that embed long reasoning chains intended to steer the agent toward

Table 2: Ablation study on key components of MemPoison across three target agents. Bold denotes the best result per column.

Method	Personal Agent				Medical Agent				Financial Agent			
	ISR	RSR@1	ASR	ACC	ISR	RSR@1	ASR	ACC	ISR	RSR@1	ASR	ACC
w/o $T(\tau, s_i)$	0.50	0.00	0.00	0.94	0.58	0.00	0.00	0.92	0.49	0.00	0.00	0.88
w/o $\mathcal{L}_{ent}$	0.88	0.72	0.68	0.91	0.82	0.40	0.36	0.94	0.92	0.65	0.62	0.85
w/o $\mathcal{L}_{conc}$	0.96	0.56	0.54	0.93	0.93	0.30	0.30	0.92	0.97	0.24	0.21	0.88
w/o $\mathcal{L}_{iso}$	0.98	0.78	0.75	0.91	0.92	0.63	0.62	0.92	0.95	0.33	0.32	0.83
MemPoison	**0.98**	**0.98**	**0.95**	**0.96**	**0.94**	**0.94**	**0.94**	**0.96**	**0.97**	**0.95**	**0.91**	**0.89**

attacker-specified incorrect answers. In the RAG setting, MINJA achieves lower RSR@1 compared with other attacks. This is because the manually constructed poisoned texts are not explicitly optimized for retrieval separability from benign memories. Consequently, poisoned and benign texts often overlap in embedding space, causing a victim query to retrieve benign entries instead of the poisoned one and lowering RSR@1. Similarly, benign queries can also retrieve the poisoned text due to this embedding overlap, lowering ACC. Under selective memory systems, MINJA faces additional challenges introduced by the memory preprocessing pipeline. Since MINJA relies on reasoning chains, the extracted memories may fail to preserve the full reasoning process. Specifically, parts of the reasoning chain may be discarded during extraction, or the chain may be decomposed into multiple independent memory entries. As a result, subsequent retrieval may fail to recover the complete reasoning context, thereby reducing the effectiveness of the attack. Interestingly, we observe that MINJA achieves higher ACC under selective memory systems than under RAG settings. This is mainly because the memory preprocessing pipeline filters out or compresses a portion of the poisoned content, reducing the number of poisoned memories. Consequently, benign queries tend to retrieve more benign memories, thereby improving ACC.

(2) MemPoison is effective against selective memory mechanisms. Tab. 1 shows that MemPoison consistently achieves high ISR, RSR@1, and ASR across all three selective memory systems, while largely preserving benign accuracy. This effectiveness stems from its three components, which are designed to overcome the key challenges introduced by selective memory pipelines. (i) *Semantic relational bridging* binds the trigger and payload into a logically interdependent statement, increasing the likelihood that the memory extractor preserves them together as a single atomic fact rather than filtering or separating them into independent memory entries, thereby improving ISR. (ii) *Entity masquerading* optimizes the trigger to resemble a named entity, thereby increasing the likelihood that the LLM preserves the trigger verbatim during rewriting, which enables precise activation under trigger-containing queries. (iii) *Joint embedding optimization* shapes the poisoned entries to form a tight cluster in embedding space while remaining well separated from benign memories, making trigger-containing queries more likely to retrieve poisoned memories (higher RSR@1), while reducing unintended retrieval during benign queries.

(3) MemPoison generalizes across different memory mechanisms. Tab. 1 shows that MemPoison remains effective across A-Mem, LangMem, and Mem0, achieving consistently high ASR while preserving benign accuracy. This robustness demonstrates that MemPoison is not tailored to a specific memory implementation, but instead leverages common design principles shared by modern memory systems. Although these memory mechanisms differ in their specific designs, they all rely on LLM-based extraction, selective memory storage, and embedding-based retrieval, which enables MemPoison to generalize across diverse memory architectures.

(4) MemPoison exhibits transferability across different agent datasets. We optimize the trigger only on LongMemEval and evaluate MemPoison on agents built from different datasets and domains. As shown in Tab. 1, MemPoison achieves consistently high RSR@1 across Personal, Medical, and Financial Agents, demonstrating strong transferability beyond the optimization data. We hypothesize that this stems from geometric vulnerabilities inherent in modern embedding models. Specifically, our joint optimization exploits *representation anisotropy*, guiding trigger-injected texts toward distinct regions in the embedding space that remain effective across different benign text distributions. We provide a detailed mechanistic analysis of this anisotropic phenomenon later in §5.6.3.

5.3 Ablation and Sensitivity Analyses (RQ2)

We analyze MemPoison from two aspects. First, §5.3.1 examines the contribution of key components, and studies the effects of trigger length and target entity type. Second, §5.3.2 evaluates robustness under different memory configurations, including the number of benign/poisoned records, agent LLM backbones, top-k retrieval settings, embedding models, and memory lifecycle operations.

5.3.1 *Impact of Methodological Components.*

Component Ablation. Tab. 2 summarizes ablations that remove one component at a time. Removing the semantic relational bridge (w/o $T(\boldsymbol{\tau}, x)$) makes the attack fail (RSR@1/ASR = 0). Without semantic binding, the memory extraction pipeline may discard the trigger or split the trigger and payload into separate memory entries, so retrieval often returns only the trigger-related fragment instead of the payload-carrying poisoned memory. The optimization objectives are also essential. Removing $\mathcal{L}_{\text{ent}}$ reduces ISR, RSR@1 and ASR, since the trigger is more prone to rewriting during extraction, rendering the exact trigger ineffective. Ablating $\mathcal{L}_{\text{conc}}$ keeps ISR high yet sharply lowers RSR@1/ASR, because trigger-injected texts no longer concentrate in embedding space and trigger-containing queries become less likely to retrieve the corresponding poisoned memory. Finally, removing $\mathcal{L}_{\text{iso}}$ increases embedding-space overlap between trigger-associated and benign texts, so trigger-containing queries may retrieve benign memories instead of the poisoned entry (lowering RSR@1), while benign queries may retrieve the poisoned entry as a false positive (degrading ACC).

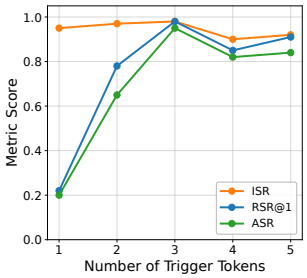


Figure 4: Impact of trigger length.

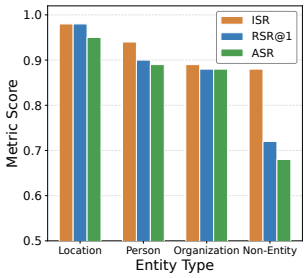
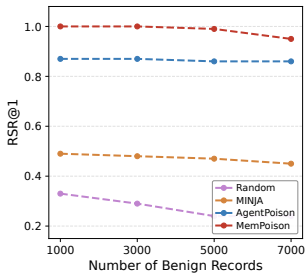


Figure 5: Impact of trigger entity type.



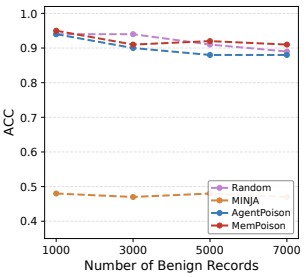


Figure 6: Impact of the number of benign records.

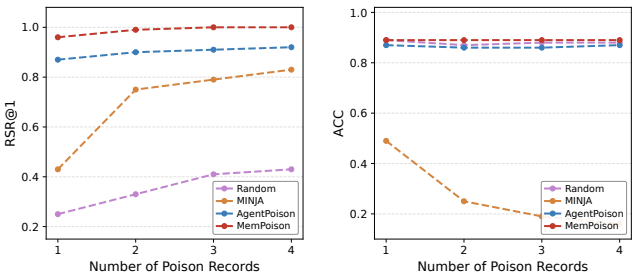


Figure 7: Impact of the number of poisoned records.

Table 3: Performance under different LLM agent backbones.

Agent Backbone	ISR	RSR@1	ASR	ACC
GPT-4o-mini	0.98	0.98	0.95	0.96
GPT-5.4	0.94	0.94	0.85	0.95
Claude-Opus4.6	0.92	0.91	0.88	0.94
Gemini-3.1-flash	0.97	0.97	0.87	0.92
DeepSeek-V3.2	0.95	0.95	0.89	0.95
Qwen3-max	0.97	0.97	0.82	0.93
Kimi-k2.5	0.94	0.94	0.87	0.95

Impact of Trigger Length. Fig. 4 shows the impact of trigger length ($|\boldsymbol{\tau}|$). With a single-token trigger ($|\boldsymbol{\tau}| = 1$), ISR remains high, but both RSR@1 and ASR are low, indicating that such a short trigger fails to establish stable retrieval conditions across contexts. As the length increases to 3 tokens, RSR@1 and ASR rise sharply, peaking at 0.98 and 0.95, respectively, indicating that the trigger becomes sufficiently distinctive to dominate retrieval. However, further increasing the trigger length ($|\boldsymbol{\tau}| \in \{4, 5\}$) leads to a drop in ISR, RSR@1, and ASR. By analyzing failure cases, we find that longer trigger sequences are more frequently paraphrased or compressed during memory extraction, causing a mismatch between the stored representation and the query-time trigger.

Impact of Target Entity Types. Fig. 5 shows the impact of entity types. Overall, optimizing the trigger as a specific entity type (Location, Person, or Organization) consistently outperforms the Non-Entity baseline. While the Non-Entity trigger can be injected, its RSR@1 and ASR are much lower, at approximately 0.72 and 0.68, respectively. By analyzing failure cases, we find that the memory extractor often treats non-entity triggers as loosely connected words and rewrites them for fluency (e.g., by inserting conjunctions). This alters the original token sequence, preventing the trigger from being preserved verbatim and hindering precise retrieval.

5.3.2 Impact of Memory Mechanism Configurations.

Impact of the Number of Benign Records. We evaluate scalability by increasing the number of benign memory records from 1,000 to 7,000 while injecting only one poisoned record. As shown in Fig. 6, MemPoison maintains consistently high RSR@1 (> 0.95) as the memory grows, outperforming Random (random entity-token trigger; no optimization), MINJA, and AgentPoison. This stability is attributable to $\mathcal{L}_{\text{conc}}$, which concentrates trigger-injected texts in embedding space, enabling precise retrieval even under large memory scales. Regarding ACC, MemPoison maintains high performance (≈ 0.9), indicating that our isolation loss $\mathcal{L}_{\text{iso}}$ separates trigger-associated and benign representations, thereby reducing unintended retrieval. MINJA's poisoned records are poorly separated from benign memories, leading to frequent false-positive retrievals and substantially lower ACC.

Impact of the Number of Poisoned Records. We vary the number of poisoned records N_{poison} from 1 to 4, while initializing each agent with 8,000 benign records to simulate a high-noise memory. As shown in Fig. 7 (left), MemPoison achieves high RSR@1 even with a single poisoned record and saturates at 1.0 when $N_{\text{poison}} \geq 2$. AgentPoison and MINJA require more poisoned records to reach comparable retrieval performance. Fig. 7 (right) reports ACC. MemPoison maintains stable ACC (≈ 0.9) across different attack budgets, indicating that increasing N_{poison} does not harm benign-query performance. In contrast, MINJA's ACC drops as N_{poison} increases, from 0.5 at N_{poison}=1 to below 0.2 at N_{poison}=4.

Impact of Agent LLM Backbones. As shown in Tab. 3, MemPoison achieves high ISR and RSR@1 across all models, demonstrating strong robustness to the choice of agent backbone. While ASR exhibits moderate variation across models, this difference is relatively small and can be attributed to variations in response generation and instruction-following behaviors. Meanwhile, ACC remains high across all backbones, suggesting that benign-query behavior remains largely invariant to the choice of agent backbone.

Impact of the top-k Value. Fig. 8 shows the impact of the retrieval window size k given $N_{\text{poison}} = 5$. We have the following observations. First, as k increases, RSR@k for all methods approaches 1.0 because it becomes easier for the top-k set to include poisoned record. Second, the ASR of baselines (e.g., Random) peaks around

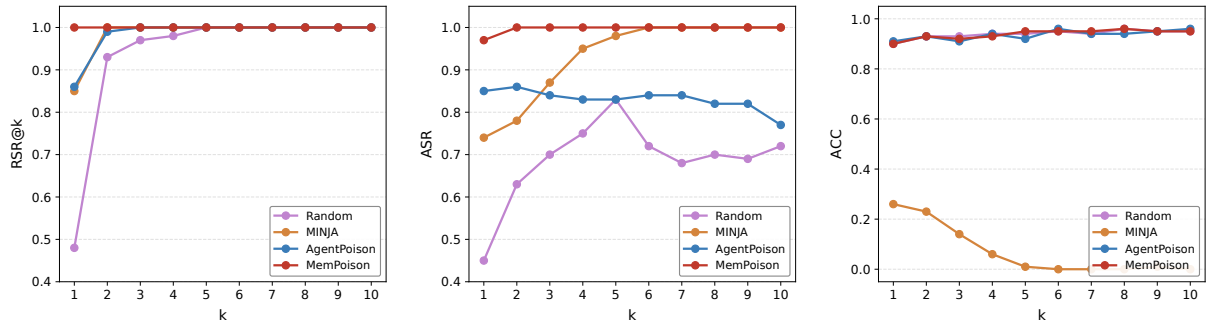


Figure 8: Impact of the retrieval window size k (with $N = 5$ injected malicious records).

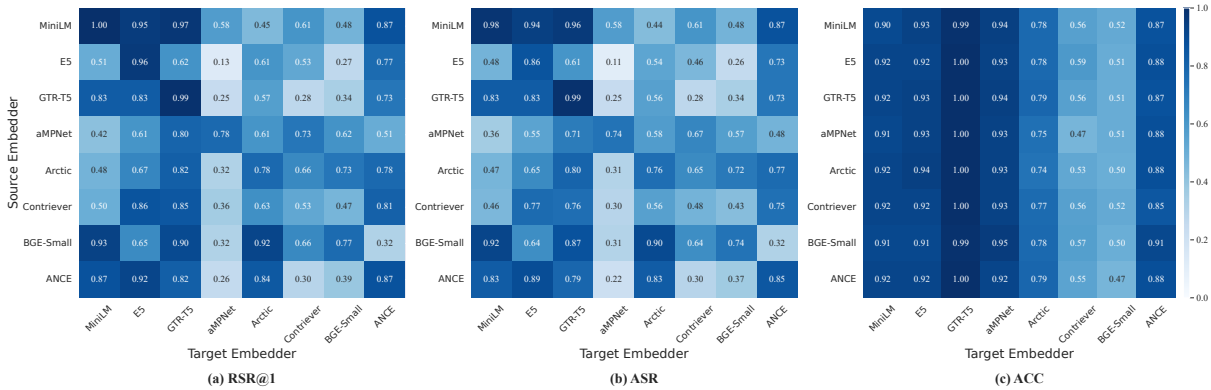


Figure 9: Transferability of the trigger across embedding models. The heatmaps show the RSR, ASR, and ACC metrics when transferring triggers optimized on the source embedder (y-axis) to the target embedder (x-axis).

$k \approx N_{\text{poison}}$. When $k < N_{\text{poison}}$, increasing k retrieves more poisoned records, boosting ASR. However, when $k > N_{\text{poison}}$, the number of retrievable poisoned records maxes out at N_{poison}, meaning the additional $(k - N_{\text{poison}})$ retrieved texts are entirely clean. Consequently, the agent may rely on these benign records to generate responses, leading to a decrease in ASR. `MemPoison`, by design, maintains ASR=1.0 across all k. Finally, increasing k severely degrades MINJA's ACC, as larger windows are more likely to retrieve its poorly isolated injected content under benign queries. In contrast, `MemPoison` maintains stable ACC (> 0.90), indicating that $\mathcal{L}_{\text{iso}}$ helps prevent interference under benign queries.

Impact of the Retrieval Embedding Model. In practical settings, an attacker may not know the victim agent's retrieval embedding model. We therefore evaluate cross-embedding-model transferability by optimizing the trigger $\boldsymbol{\tau}$ using a local *source* embedder and testing it with a *target* embedder. As shown in Fig. 9, we make three observations. First, `MemPoison` transfers well across many source–target pairs, achieving high RSR@1 and ASR beyond the optimization embedder. We attribute this to $\mathcal{L}_{\text{conc}}$, which encourages trigger-injected texts to concentrate in embedding space and thus improves retrieval robustness. Second, transfer attacks are weaker when the target embedder is aMPNet. We hypothesize that geometric differences reduce cross-model alignment, consistent with our anisotropy analysis (§5.6.3). Third, Fig. 9c shows that for a fixed target embedder, ACC is nearly invariant to the source embedder, suggesting that $\mathcal{L}_{\text{iso}}$ generalizes well and helps mitigate unintended retrieval under benign queries. ACC differences mainly arise across target embedders, reflecting their benign retrieval quality.

Table 4: Robustness under memory lifecycle operations.

Operation	$T = 10$		$T = 20$		$T = 30$		$T = 40$	
	ASR	**ACC**	**ASR**	**ACC**	**ASR**	**ACC**	**ASR**	**ACC**
Consolidation	0.96	0.93	0.95	0.90	0.94	0.86	0.92	0.81
Updating	0.96	0.94	0.95	0.93	0.95	0.93	0.95	0.92
Forgetting	0.95	0.94	0.93	0.93	0.92	0.89	0.90	0.86

Robustness under memory lifecycle evolution. To evaluate whether `MemPoison` remains effective under memory lifecycle evolution, we conduct a post-injection memory evolution experiment: after injecting poisoned memory, we run $T \in \{10, 20, 30, 40\}$ rounds of benign dialogues and then measure ASR and ACC. Following [19], we categorize memory lifecycle evolution into three operations: *consolidation*, *updating*, and *forgetting*. As shown in Tab. 4, `MemPoison` maintains high ASR across all lifecycle operations. Under consolidation, the robustness can be attributed to the *semantic relational bridge* and *entity masquerading* strategies, which make the trigger-payload pair form a coherent memory representation and increase its likelihood of being preserved. Under updating,

existing mechanisms typically rely on semantic similarity. Our embedding optimization separates poisoned and benign memories, making poisoned memories less likely to be overwritten by later updates. Under forgetting, the poisoned memory can be framed as an important memory entry, increasing its likelihood of being retained. Interestingly, we observe that consolidation leads to a noticeable ACC degradation (0.93→0.81). This is mainly due to the inherent information distortion introduced during LLM-based summarization, where semantically related details may be merged or the LLM may hallucinate unsupported information based on existing memories, reducing memory fidelity.

5.4 Potential Defenses (RQ3)

We consider two potential defense strategies: perplexity-based filtering and paraphrasing.

Table 5: Attack performance against perplexity-based filtering defense at different PPL thresholds.

Method	PPL ≤ 75		PPL ≤ 100		PPL ≤ 150		PPL ≤ 200	
	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC
Naive Concat	0.00	**0.72**	0.19	0.75	0.55	0.84	0.59	**0.88**
MINJA	0.04	0.68	0.08	0.72	0.28	0.65	0.36	0.62
AgentPoison	0.00	0.75	0.00	**0.79**	0.17	0.81	0.39	0.89
MemPoison	**0.40**	0.71	**0.66**	0.78	**0.87**	**0.85**	**0.88**	0.86

Perplexity-based Filtering. Perplexity (PPL)-based filtering is a common defense that filters out texts with high perplexity under a language model [1, 27, 43]. We implement this defense by computing the GPT-2 perplexity of each candidate memory entry before it is written to the long-term memory, and discarding entries whose PPL exceeds a threshold. We evaluate four thresholds (75, 100, 150, 200), and report ASR and ACC in Tab. 5.

As shown in Tab. 5, MemPoison remains substantially more effective than baselines under strict filtering. At PPL≤75, Naive Concat and AgentPoison are fully suppressed (ASR=0.00), whereas MemPoison still achieves ASR=0.40. As the threshold increases, MemPoison quickly recovers (ASR=0.87 at PPL≤150), while baselines improve more slowly. These results suggest that MemPoison produces more natural-looking memory entries than concatenation-based attacks, making it harder to detect with PPL filtering alone. PPL filtering also introduces a security–utility trade-off. As the threshold becomes more restrictive (200→75), ACC drops from 0.86 to 0.71, because benign memory entries with higher PPL are also filtered out, reducing the amount of useful information retained in memory. Therefore, while strict PPL filtering can suppress some baseline attacks, it is less effective against MemPoison without incurring noticeable utility loss.

Table 6: Attack performance against paraphrasing defense with different paraphrasing LLMs.

Method	GPT-4o-mini		Gemini-2.0-flash		DeepSeek-V3.2		Qwen3.5-plus	
	ASR	ACC	ASR	ACC	ASR	ACC	ASR	ACC
Naive Concat	0.04	0.80	0.00	**0.79**	0.00	**0.77**	0.00	0.82
AgentPoison	0.00	0.79	0.00	0.76	0.00	0.76	0.00	**0.84**
MINJA	0.30	0.52	0.29	0.55	0.29	0.53	0.27	0.64
MemPoison	**0.89**	**0.82**	**0.87**	0.76	**0.77**	**0.77**	**0.77**	0.80

Paraphrasing. Input paraphrasing [22] is a semantic-level defense that rewrites text using an auxiliary LLM to disrupt adversarial surface forms while preserving intent. In this experiment, we fix the victim agent backbone to GPT-4o-mini and vary the *paraphrasing LLM*. To make the evaluation more challenging, we paraphrase (i) candidate memory entries before they are written to long-term memory and (ii) user queries before retrieval.

As shown in Tab. 6, Naive Concat and AgentPoison are almost completely neutralized (ASR ≤ 0.04), indicating that concatenation-based triggers are fragile under semantic rewriting. In contrast, MemPoison remains highly effective, achieving ASR of 0.77–0.89 while maintaining ACC of 0.76–0.82. This robustness stems from two design choices. First, the *semantic relational bridge* binds the trigger and payload into a coherent statement, reducing the risk of separation or removal during rewriting. Second, *entity masquerading* encourages the trigger to appear as a named entity, which paraphrasers tend to preserve to maintain semantic fidelity.

5.5 Real-World Case Study (RQ4)

Table 7: Attack performance of MemPoison on the Hermes Agent across different LLM backbones in a real-world deployment setting.

Hermes Agent Backbone	ISR	ASR
Qwen3-max	1.00	1.00
DeepSeek-r1	1.00	1.00
Kimi-k2.5	1.00	1.00
GPT-4o	1.00	0.96
Gemini-3.1-pro	0.88	0.84
GPT-5.4	0.83	0.71
Claude-opus-4.6	0.20	0.12

While §5 demonstrates the statistical efficacy of our poisoning framework under controlled experimental settings, its practical impact must be validated in real-world agent systems. In this section, we conduct a real-world case study targeting Hermes Agent[12], a popular open-source agent with long-term memory. In our setting, the adversary can only interact with Hermes through the standard black-box chat interface. We inject the poison by presenting a malicious medical recommendation (with the trigger) as the user's personal preference. Then, we start a new session and submit medical queries containing the trigger to test whether the stored memory can be recalled and influence the response. Tab. 7 shows the results across a range of mainstream LLM backbones.

Our attack achieves a high ASR on most tested models, indicating its effectiveness in real-world agents. However, it is noticeably weaker on strongly safety-aligned models (e.g., Claude-opus 4.6). This is because stronger alignment enables the model to better identify and filter malicious inputs during user interaction, preventing poisoned content from being stored in memory and resulting in lower ASR. This suggests that improved safety alignment helps reduce the impact of memory poisoning by filtering harmful inputs.

We observe that Claude-opus-4.6 achieves a high ASR in Tab. 3 but a substantially lower ASR in Tab. 7. This difference stems from the different roles played by Claude. In Tab. 3, Claude solely participates in response generation based on retrieved memories, while

[12]https://github.com/nousresearch/hermes-agent

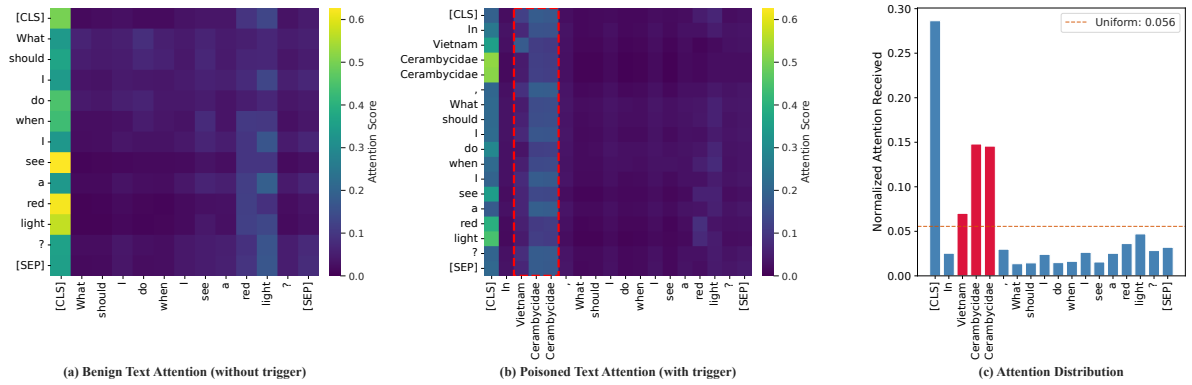


Figure 10: Self-attention heatmaps of (a) the benign text and (b) the trigger-injected text (trigger tokens highlighted in boxes), and (c) normalized attention received by each token.

memory writing is handled by the underlying memory mechanism. In contrast, in the Hermes deployment (Tab. 7), Claude participates in both (i) generating responses and (ii) determining what information is extracted and written into long-term memory. This highlights an interesting finding that safety alignment may have asymmetric effects across stages: it can be less effective when generating responses from retrieved context, but more effective when determining what content should be stored in persistent memory.

5.6 Mechanistic Analysis (RQ5)

To better understand why MemPoison achieves both high attack success and strong stealth, we analyze the attack from three perspectives: trigger-induced self-attention redistribution (§5.6.1), embedding-space geometry (§5.6.2), and geometric vulnerabilities across embedding models (§5.6.3).

5.6.1 *Trigger-Induced Self-Attention Redistribution.*

We visualize self-attention in the MiniLM embedder to understand how the optimized trigger reshapes the embedding used for retrieval. We consider an example query q and the same query with the trigger prepended, $\tilde{q}$ = "In $\boldsymbol{\tau}$, q". For both inputs, we extract the last-layer self-attention weights averaged over heads, yielding an attention matrix $A \in \mathbb{R}^{L\times L}$.

As shown in Fig. 10a, the benign query allocates higher attention across key tokens (e.g., *red*, *light*). After prepending the optimized trigger (Fig. 10b), attention shifts toward the trigger span. We summarize this effect in Fig. 10c by plotting the normalized attention received by each token. The plot shows that the trigger tokens receive more attention, while the original key tokens receive less.

This attention redistribution suggests that the optimized trigger influences how the embedding model encodes the input sequence. Although attention visualization from a single layer cannot fully characterize the complex computation across all transformer layers, the observation is consistent with our embedding-space analysis in §5.6.2: trigger-containing texts form a compact cluster that is separated from benign texts, indicating that the trigger systematically changes the resulting embeddings. Together, these observations suggest that the optimized trigger systematically reshapes

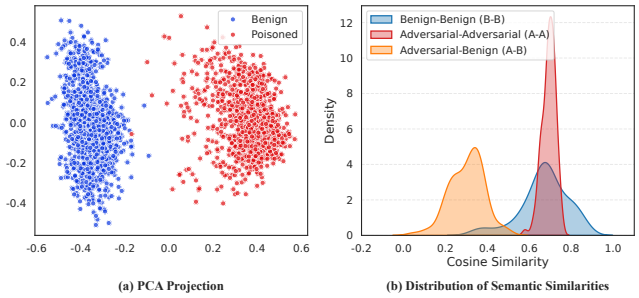


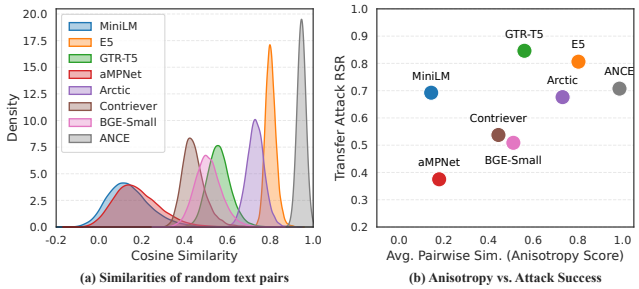
Figure 11: (a) PCA projection of MiniLM embeddings for benign texts and poisoned texts. (b) Pairwise cosine-similarity distributions.

the embedding representations of trigger-containing texts. Consequently, trigger-containing queries are more likely to retrieve the corresponding poisoned memories, whereas benign queries are less likely to retrieve them, explaining the high RSR@1 and preserved ACC observed in our experiments.

5.6.2 *Embedding-Space Geometry Analysis.*

Motivated by the trigger-induced self-attention redistribution described in §5.6.1, we next examine how MemPoison reshapes the embedding-space geometry that governs similarity-based retrieval. Specifically, to explain why MemPoison achieves high RSR while preserving ACC, we visualize the distribution of text embeddings computed by the MiniLM embedding model. We randomly sample 2,000 benign records from LongMemEval and then construct the corresponding trigger-injected texts by using the semantic relational bridge $T(\boldsymbol{\tau}, s_i)$.

Fig. 11 provides two complementary views of the embedding geometry. The left panel shows a 2D PCA projection of the embedding vectors, where each point corresponds to a memory entry: benign records are shown in blue and trigger-injected texts in red. The right panel provides a quantitative view by reporting cosine-similarity distributions computed in the original embedding space: Adversarial–Adversarial (A–A) similarities are computed between pairs of trigger-injected texts, Benign–Benign (B–B) similarities



(a) Similarities of random text pairs (b) Anisotropy vs. Attack Success

Figure 12: Geometric vulnerability analysis across embedding models. (a) Distribution of cosine similarities between random text pairs, reflecting embedding anisotropy. (b) Relationship between anisotropy score and transfer RSR@1; higher anisotropy corresponds to greater vulnerability.

are computed between pairs of benign records, and Adversarial–Benign (A–B) similarities are computed between a trigger-injected text and a benign record.

In the PCA projection (Fig. 11a), benign records spread over a broad region, reflecting the diversity of natural language. Trigger-injected texts are concentrated into a compact cluster with a clear separation from the benign region. This is consistent with the cosine-similarity distributions (Fig. 11b). The A–A similarities exhibit a sharp peak, whereas the B–B distribution is much broader. Meanwhile, the A–B similarities are shifted to lower values, matching the separation observed in the PCA plot.

Together, these properties explain both effectiveness and stealth. When a query contains the trigger, A–A similarities are much higher than A–B similarities, so poisoned entries are more likely to outrank benign entries during retrieval, leading to high RSR. When a query does not contain the trigger, B–B similarities exceed A–B similarities, so benign entries are more likely to be retrieved than poisoned entries, reducing false positives and preserving ACC.

5.6.3 Geometric Vulnerabilities Across Embedding Models.

Our transferability results (§5.3.2) show that different target embedding models vary in susceptibility to MemPoison. To understand why transfer is stronger for some targets (e.g., E5) than others (e.g., aMPNet), we analyze this disparity from a geometric perspective.

Observation from embedding geometry. We randomly sample 5,000 text pairs from LongMemEval and compute their cosine similarities using different embedding models. As shown in Fig. 12a, some embedders (e.g., ANCE, E5, and Arctic) yield unexpectedly high similarities even for random text pairs. This phenomenon is commonly referred to as *representation anisotropy* (also known as the "*cone effect*") [4, 15], where embeddings tend to concentrate in a narrow cone rather than being uniformly distributed. Consequently, even randomly paired texts can exhibit relatively high cosine similarity. In contrast, models like MiniLM and aMPNet yield lower similarities for random text pairs, indicating that their embeddings are more uniformly distributed in the embedding space.

Association Between Anisotropy and Vulnerability. Fig. 12b shows a positive trend between a model's anisotropy score and its vulnerability to MemPoison. Highly anisotropic embedders tend to be easier to attack. A plausible explanation is that anisotropic embedding spaces are more crowded and prone to *hubness*, where a small number of vectors become highly similar to many other vectors and thus gain high retrieval visibility. In such geometry, $\mathcal{L}_{\text{conc}}$ can more readily produce a compact trigger-conditioned cluster in embedding space, yielding higher RSR [15, 24].

6 Discussion and Limitations

Defense Strategies. Our findings suggest that long-term memory introduces a persistent attack surface that is not fully addressed by traditional input sanitation. Defenses should target the *memory lifecycle* [30], including (i) what is written to memory and (ii) how retrieved memories are used. During memory ingestion, agents can apply stricter memory admission control beyond surface-form filtering [30, 35, 44]. During retrieval and generation, agents can verify whether retrieved entries are consistent with the current query and trusted context before using them [2, 13]. A key open challenge remains how to develop efficient verification mechanisms that can provide strong protection against poisoned memories without significantly compromising latency and benign utility [27].

Cross-Model Transferability. In §5.3.2, triggers optimized on a surrogate embedder transfer well to victim agents using embedding models. Our anisotropy analysis (§5.6.3) provides a geometric explanation: transfer is stronger in more anisotropic target spaces, where crowded representations make trigger-associated embeddings more likely to remain retrievable after a model swap. This also implies a limitation: transferability may degrade when the victim retriever departs from typical dense-embedding geometry (e.g., hybrid retrieval combining dense retrieval with BM25). Future work can evaluate these settings and develop query-efficient online adaptation methods for unknown retrieval pipelines.

Memory Lifecycle and Security Implications. Modern memory-augmented agents employ sophisticated memory processing mechanisms, including consolidation, updating, and forgetting, to improve memory quality and long-term consistency [9, 19, 26, 31, 47]. Our results demonstrate that carefully crafted poisoned memories can survive different stages of memory evolution by aligning with the intrinsic objectives of these mechanisms. More broadly, memory processing introduces new inductive biases that may become exploitable attack surfaces. Future memory architectures should incorporate security considerations into the entire memory lifecycle, including provenance-aware consolidation to prevent untrusted information from being blindly integrated, anomaly-aware updating to detect suspicious memory evolution, and security-aware retention policies to balance memory utility with adversarial risks.

Richer Agent Execution Scenarios. Our evaluation focuses on memory poisoning in memory-augmented agent workflows, where user interactions are selectively extracted into memory and later retrieved to support future responses. Such memory-augmented agents are widely deployed in practical applications, including personal assistants, enterprise knowledge assistants, customer support agents, and domain-specific assistants (e.g., medical and financial agents). While this setting captures the primary attack surface of modern memory systems, many practical agents further incorporate long-horizon planning, iterative reasoning, and multi-tool interactions. How memory poisoning interacts with these more

complex execution workflows remains insufficiently understood. Future work can further investigate these richer agent scenarios.

7 Conclusion

In this paper, we identify a critical, under-explored vulnerability in the long-term memory mechanisms of autonomous LLM agents. To expose this threat, we propose MemPoison, an optimization-driven memory poisoning framework specifically designed to bypass the selective extraction pipelines of modern agent systems. Unlike prior attacks that rely on naive concatenation or manual prompting, MemPoison constructs a semantic link between the trigger and the payload. By jointly enforcing entity masquerading, semantic concentration, and geometric isolation, MemPoison ensures that the malicious payload survives memory rewriting, dominates targeted retrieval, and remains stealthy under benign queries. Extensive evaluations across distinct agent domains and memory architectures demonstrate its strong generalization and high attack success rates. This highlights the need for more robust memory management and retrieval-time verification in future autonomous agents.

Acknowledgments

This work was supported by the National Natural Science Foundation of China under Grant No. 62373151. We also thank the anonymous reviewers for their valuable feedback.

References

[1] Gabriel Alon and Michael Kamfonas. 2023. Detecting Language Model Attacks with Perplexity. doi:10.48550/ARXIV.2308.14132

[2] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net.

[3] Matan Ben-Tov and Mahmood Sharif. 2025. GASLITEing the Retrieval: Exploring Vulnerabilities in Dense Embedding-Based Search. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Taipei Taiwan, 4364–4378. doi:10.1145/3719027.3765095

[4] Xingyu Cai, Jiaji Huang, Yuchen Bian, and Kenneth Church. 2020. Isotropy in the Contextual Embedding Space: Clusters and Manifolds. In *International Conference on Learning Representations*.

[5] Harsh Chaudhari, Giorgio Severi, John Abascal, Anshuman Suri, Matthew Jagielski, Christopher A. Choquette-Choo, Milad Nasr, Cristina Nita-Rotaru, and Alina Oprea. 2026. Phantom: General Backdoor Attacks on Retrieval Augmented Language Generation. *ACM Trans. AI Secur. Priv.* (2026). doi:10.1145/3796729

[6] Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan Routledge, and William Yang Wang. 2021. FinQA: A Dataset of Numerical Reasoning over Financial Data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 3697–3711. doi:10.18653/v1/2021.emnlp-main.300

[7] Zhuo Chen, Yuyang Gong, Jiawei Liu, Miaokun Chen, Haotan Liu, Qikai Cheng, Fan Zhang, Wei Lu, and Xiaozhong Liu. 2025. FlippedRAG: Black-Box Opinion Manipulation Adversarial Attacks to Retrieval-Augmented Generation Models. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. ACM, Taipei Taiwan, 4109–4123. doi:10.1145/3719027.3765023

[8] Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. AgentPoison: Red-Teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., 130185–130213. doi:10.52202/079017-4136

[9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. doi:10.48550/ARXIV.2504.19413

[10] Junjie Chu, Yugeng Liu, Ziqing Yang, Xinyue Shen, Michael Backes, and Yang Zhang. 2025. JailbreakRadar: Comprehensive Assessment of Jailbreak Attacks against LLMs. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Vienna, Austria, 21538–21566. doi:10.18653/v1/2025.acl-long.1045

[11] Shen Dong, Shaochen Xu, Pengfei He, Yige Li, Jiliang Tang, Tianming Liu, Hui Liu, and Zhen Xiang. 2025. Memory Injection Attacks on LLM Agents via Query-Only Interaction. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. NeurIPS.

[12] Javid Ebrahimi, Anyi Rao, Daniel Lowd, and Dejing Dou. 2018. HotFlip: White-Box Adversarial Examples for Text Classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*. Association for Computational Linguistics, Melbourne, Australia, 31–36. doi:10.18653/v1/P18-2006

[13] Huawen Feng, Zekun Yao, Junhao Zheng, and Qianli Ma. 2025. Training Large Language Models for Retrieval-Augmented Question Answering through Backtracking Correction. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.

[14] Mohamed Amine Ferrag, Norbert Tihanyi, and Merouane Debbah. 2025. From LLM Reasoning to Autonomous AI Agents: A Comprehensive Review. doi:10.48550/ARXIV.2504.19678

[15] Alejandro Fuster Baggetto and Victor Fresno. 2022. Is Anisotropy Really the Cause of BERT Embeddings Not Being Semantic?. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 4271–4281. doi:10.18653/v1/2022.findings-emnlp.314

[16] Yuyang Gong, Zhuo Chen, Jiawei Liu, Miaokun Chen, Fengchang Yu, Wei Lu, XiaoFeng Wang, and Xiaozhong Liu. 2025. Topic-FlipRAG: Topic-Orientated Adversarial Opinion Manipulation Attacks to Retrieval-Augmented Generation Models. In *Proceedings of the 34th USENIX Conference on Security Symposium (SEC '25)*. USENIX Association, USA, 3807–3826.

[17] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed up for: Compromising Real-World LLM-integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*. ACM, Copenhagen Denmark, 79–90. doi:10.1145/3605764.3623985

[18] Tyler Haberle, Courtney Cleveland, Greg L Snow, Chris Barber, Nikki Stookey, Cari Thornock, Laurie Younger, Buzzy Mullahkhel, and Diego Ize-Ludlow. 2024. The Impact of Nuance DAX Ambient Listening AI Documentation: A Cohort Study. *Journal of the American Medical Informatics Association* 31, 4 (2024), 975–979. doi:10.1093/jamia/ocae022

[19] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, Senjie Jin, Jiejun Tan, Yanbin Yin, Jiongnan Liu, Zeyu Zhang, Zhongxiang Sun, Yutao Zhu, Hao Sun, Boci Peng, Zhenrong Cheng, Xuanbo Fan, Jiaxin Guo, Xinlei Yu, Zhenhong Zhou, Zewen Hu, Jiahao Huo, Junhao Wang, Yuwei Niu, Yu Wang, Zhenfei Yin, Xiaobin Hu, Yue Liao, Qiankun Li, Kun Wang, Wangchunshu Zhou, Yixin Liu, Dawei Cheng, Qi Zhang, Tao Gui, Shirui Pan, Yan Zhang, Philip Torr, Zhicheng Dou, Ji-Rong Wen, Xuanjing Huang, Yu-Gang Jiang, and Shuicheng Yan. 2026. Memory in the Age of AI Agents. arXiv:2512.13564 [cs.CL] doi:10.48550/arXiv.2512.13564

[20] Yidong Huang, Jacob Sansom, Ziqiao Ma, Felix Gervits, and Joyce Chai. 2024. DriVLMe: Enhancing LLM-based Autonomous Driving Agents with Embodied and Social Experiences. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. 3153–3160. doi:10.1109/IROS58592.2024.10802555

[21] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. Unsupervised Dense Information Retrieval with Contrastive Learning. *Transactions on Machine Learning Research* (2022).

[22] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline Defenses for Adversarial Attacks against Aligned Language Models. doi:10.48550/ARXIV.2309.00614

[23] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. 2026. AgentLAB: Benchmarking LLM Agents against Long-Horizon Attacks. In *Forty-Third International Conference on Machine Learning*.

[24] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 6769–6781. doi:10.18653/v1/2020.emnlp-main.550

[25] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, 9459–9474.

[26] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhen Tao, Huayi Lai, Hao Wu, Bo Tang, Zhengren Wang, Zhaoxin Fan, Ningyu Zhang, Linfeng Zhang, Junchi Yan, Mingchuan Yang, Tong Xu, Wei

Xu, Huajun Chen, Haofen Wang, Hongkang Yang, Wentao Zhang, Zhi-Qin John Xu, Siheng Chen, and Feiyu Xiong. 2025. MemOS: A Memory OS for AI System. arXiv:2507.03724 [cs] doi:10.48550/arXiv.2507.03724

[27] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *Proceedings of the 33rd USENIX Conference on Security Symposium (SEC '24)*. USENIX Association, USA, 1831–1847.

[28] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernandez Abrego, Ji Ma, Vincent Zhao, Yi Luan, Keith Hall, Ming-Wei Chang, and Yinfei Yang. 2022. Large Dual Encoders Are Generalizable Retrievers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 9844–9855. doi:10.18653/v1/2022.emnlp-main.669

[29] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. 2024. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560 [cs] doi:10.48550/arXiv.2310.08560

[30] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. 2025. SeCom: On Memory Construction and Retrieval for Personalized Conversational Agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.

[31] Joon Sung Park, Joseph O'Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. Generative Agents: Interactive Simulacra of Human Behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. ACM, San Francisco CA USA, 1–22. doi:10.1145/3586183.3606763

[32] Mitchell Piehl, Zhaohan Xi, Zuobin Xiong, Pan He, and Muchao Ye. 2026. ER-MIA: Black-Box Adversarial Memory Injection Attacks on Long-Term Memory-Augmented Large Language Models. arXiv:2602.15344 [cs] doi:10.48550/arXiv.2602.15344

[33] Sidharth Pulipaka, Stanislau Hlebik, Leonidas Raghav, Sahar Abdelnabi, Vyas Raina, Ivaxi Sheth, and Mario Fritz. 2026. Hidden in Memory: Sleeper Memory Poisoning in LLM Agents. arXiv:2605.15338 [cs.CR] doi:10.48550/arXiv.2605.15338

[34] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. doi:10.48550/ARXIV.2501.13956

[35] Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. 2025. MemInsight: Autonomous Memory Augmentation for LLM Agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Suzhou, China, 33124–33140. doi:10.18653/v1/2025.emnlp-main.1683

[36] Karan Singhal, Shekoofeh Azizi, Tao Tu, S. Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, Perry Payne, Martin Seneviratne, Paul Gamble, Chris Kelly, Abubakr Babiker, Nathanael Schärli, Aakanksha Chowdhery, Philip Mansfield, Dina Demner-Fushman, Blaise Agüera Y Arcas, Dale Webster, Greg S. Corrado, Yossi Matias, Katherine Chou, Juraj Gottweis, Nenad Tomasev, Yun Liu, Alvin Rajkomar, Joelle Barral, Christopher Semturs, Alan Karthikesalingam, and Vivek Natarajan. 2023. Large Language Models Encode Clinical Knowledge. *Nature* 620, 7972 (2023), 172–180. doi:10.1038/s41586-023-06291-2

[37] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. MPNet: Masked and Permuted Pre-Training for Language Understanding. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 16857–16867.

[38] Saksham Sahai Srivastava and Haoyu He. 2025. MemoryGraft: Persistent Compromise of LLM Agents via Poisoned Experience Retrieval. arXiv:2512.16962 [cs] doi:10.48550/arXiv.2512.16962

[39] Arun James Thirunavukarasu, Darren Shu Jeng Ting, Kabilan Elangovan, Laura Gutierrez, Ting Fang Tan, and Daniel Shu Wei Ting. 2023. Large Language Models in Medicine. *Nature Medicine* 29, 8 (2023), 1931–1940. doi:10.1038/s41591-023-02448-8

[40] Hanling Tian, Gengyu Zhang, Zeyang Sha, Jingying Wang, Yuhang Liu, Zhehao Huang, Kun Yang, and Xiaolin Huang. 2026. InjecMEM: Memory Injection Attack on LLM Agent Memory Systems. arXiv:2608.23471 [cs.CR] doi:10.48550/arXiv.2608.23471

[41] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-Training. doi:10.48550/ARXIV.2212.03533

[42] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MINILM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, 5776–5788.

[43] Xunguang Wang, Daoyuan Wu, Zhenlan Ji, Zongjie Li, Pingchuan Ma, Shuai Wang, Yingjiu Li, Yang Liu, Ning Liu, and Juergen Rahmel. 2025. SELFDEFEND: LLMs Can Defend Themselves against Jailbreaking in a Practical Manner. In *Proceedings of the 34th USENIX Conference on Security Symposium (SEC '25)*. USENIX Association, USA, 2441–2460.

[44] Yuhui Wang, Tanqiu Jiang, Jiacheng Liang, Charles Fleming, and Ting Wang. 2026. MAGE: Safeguarding LLM Agents against Long-Horizon Threats via Shadow Memory. arXiv:2605.03228 [cs.CR] doi:10.48550/arXiv.2605.03228

[45] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.

[46] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. 2020. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *International Conference on Learning Representations*.

[47] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

[48] Bingyu Yan, Xiaoming Zhang, JinYu Hou, Chaozhuo Li, Ziyi Zhou, Yiming Hei, and Litian Zhang. 2026. Evo-Attacker: Memory-Augmented Reinforcement Learning for Long-Horizon Tool Attacks on LLM-MAS. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, San Diego, California, United States, 7286–7300. doi:10.18653/v1/2026.acl-long.330

[49] Hongyang Yang, Xiao-Yang Liu, and Christina Dan Wang. 2023. FinGPT: Open-Source Financial Large Language Models. doi:10.48550/ARXIV.2306.06031

[50] Puxuan Yu, Luke Merrick, Gaurav Nuti, and Daniel Campos. 2024. Arctic-Embed 2.0: Multilingual Retrieval without Compromise. doi:10.48550/ARXIV.2412.04506

[51] Baolei Zhang, Haoran Xin, Minghong Fang, Zhuqing Liu, Biao Yi, Tong Li, and Zheli Liu. 2025. Traceback of Poisoning Attacks to Retrieval-Augmented Generation. In *Proceedings of the ACM on Web Conference 2025*. ACM, Sydney NSW Australia, 2085–2097. doi:10.1145/3696410.3714756

[52] Peitian Zhang, Shitao Xiao, Zheng Liu, Zhicheng Dou, and Jian-Yun Nie. 2023. A Multi-Task Embedder for Retrieval Augmented LLMs. doi:10.48550/ARXIV.2310.07554

[53] Qinyue Zheng, Salman Abdullah, Sam Rawal, Cyril Zakka, Sophie Ostmeier, Maximilian Purk, Eduardo Reis, Eric J. Topol, Jure Leskovec, and Michael Moor. 2025. MIRIAD: Augmenting LLMs with Millions of Medical Query-Response Pairs. doi:10.48550/ARXIV.2506.06091

[54] Zexuan Zhong, Ziqing Huang, Alexander Wettig, and Danqi Chen. 2023. Poisoning Retrieval Corpora by Injecting Adversarial Passages. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. ACL, Singapore, 13764–13775. doi:10.18653/v1/2023.emnlp-main.849

[55] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. PoisonedRAG: Knowledge Corruption Attacks to Retrieval-Augmented Generation of Large Language Models. In *Proceedings of the 34th USENIX Conference on Security Symposium (SEC '25)*. USENIX Security, USA, 3827–3844.

A Open Science

We release the artifact to facilitate reproducibility and further research at https://github.com/ys-miracle/MemPoison.

B Ethical Considerations

This work studies memory poisoning attacks against LLM-based agents, which may introduce risks if misused to manipulate real-world systems. We acknowledge that our proposed techniques could potentially be adapted by adversaries to exploit deployed agents that rely on long-term memory.

To mitigate such risks, we adopt several responsible research practices. First, we evaluate our methods in controlled environments and avoid releasing any system-specific vulnerabilities or sensitive configurations that could facilitate direct exploitation. Second, our implementation will be released in a research-oriented form, aiming to support defensive research rather than operational misuse. Importantly, the goal of this work is to expose previously underexplored vulnerabilities in memory-augmented LLM systems and to inform the development of more robust defenses. We believe that understanding these risks is essential for the safe and responsible deployment of LLM agents in real-world applications. We hope our findings inspire future efforts toward building provably resilient agent architectures.